Javascript

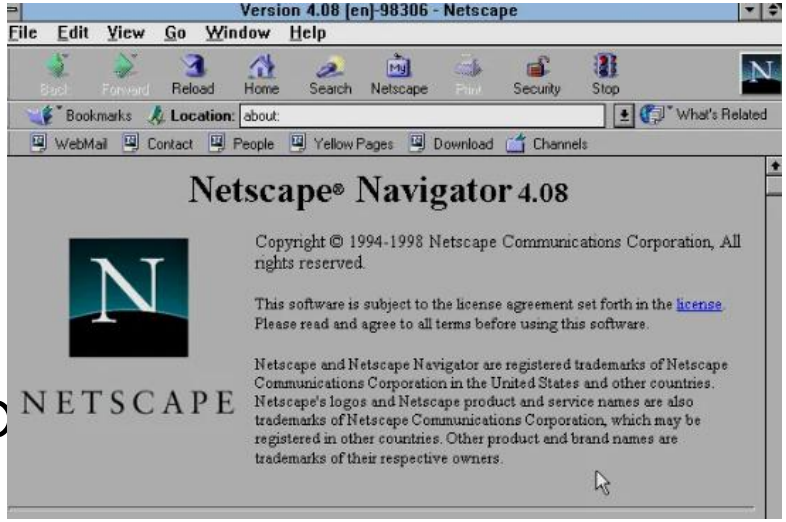
MBP 214 Web Programlama

Bahar 25-26

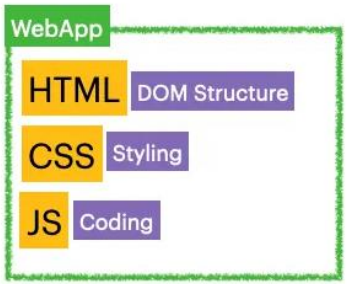
İEU Bilgisayar Prog.

JavaScript

- Javascript , Netscape firması tarafından C dilinden esinlenilerek yazılmıştır.
  LiveScript & Netscape Navigator
- Yazılma amacı Html'in sahip olmadığı bazı özelliklerin web sayfalarında kullanılmak istenmesidir
- Java programlama dilinden tamamen farklıdır karıştırmayınız. JavaScript sadece isim babası



JavaScript Amacı

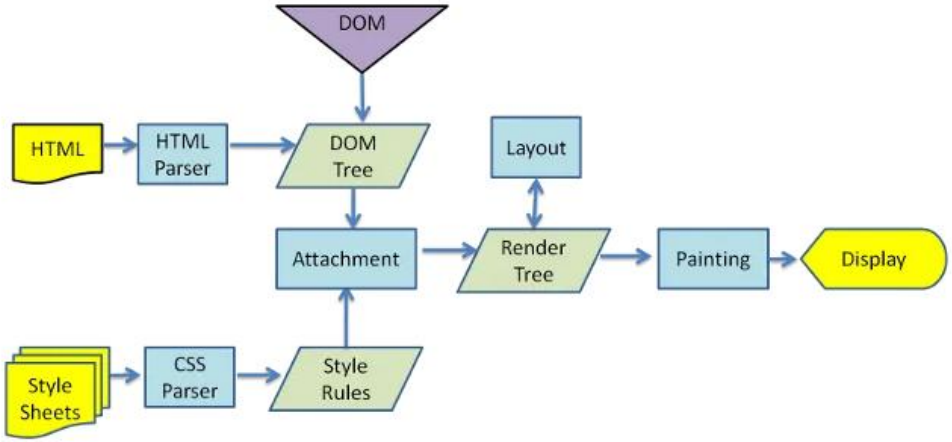


JavaScript

Web Sayfası = İçerik (HTML) + Sunum (CSS) + Etkileşim (JS)

- **Etkileşim:** Kullanıcı bir butona tıkladığında formun onaylanması, açılır menülerin çalışması veya bir "beğen" butonunun anında renk değiştirmesi JS ile olur.
- **Dinamik İçerik:** Sayfayı yenilemeden verilerin güncellenmesini sağlar (örneğin, canlı skorlar veya mesajlaşma uygulamaları).
- **Hız:** İşlemlerin çoğu kullanıcının bilgisayarında (istemci tarafında) gerçekleştiği için sunucu yükünü azaltır ve hızı artırır.
- **Modern API'ler:** Coğrafi konum (GPS), kamera erişimi ve ses işleme gibi gelişmiş özellikleri tarayıcıya taşır.

Web Tarayıcıda Nasıl Render Edilir?



1. **Ayrıştırma (Parsing):** Tarayıcı HTML'i okuyarak DOM ağacını, CSS'i okuyarak CSSOM ağacını oluşturur.
2. **JS Engine Devreye Girer:** Tarayıcının içindeki JavaScript motoru (örneğin Chrome'daki V8 motoru) kodu satır satır okur.
3. **Engelleme (Blocking):** Varsayılan olarak, tarayıcı bir <script> etiketiyle karşılaştığında HTML'i okumayı durdurur, JS'yi indirir ve çalıştırır. (*Bu yüzden performans için JS dosyaları genellikle sayfanın en altına konur*).
4. **İşleme Ağacı (Render Tree):** DOM ve CSSOM birleşerek "Render Tree"yi oluşturur. JS bu ağaç üzerinde bir değişiklik yaparsa (örneğin bir elementi gizlerse), tarayıcı sayfayı yeniden çizer (Repaint veya Reflow).

Nasıl Kullanılır?

Dahili (internal)

- <html> içinde <script> taglerine yazılarak

Harici (external)

- .js uzantılı bir dosya oluşturup HTML'e bağlanarak.

<script src="script.js"></script>

Harici Kullanım - JavaScript Kodlarını Kaynak Dosyada Saklamanın Avantajları

- **Kolay Bakım ve Güncelleme**: Eğer aynı kodlar ana sayfa ve tüm alt sayfalarda kullanılıyorsa, kaynak dosya kullanmadığınızda bir değişiklik yapmak için her sayfadaki kodu tek tek düzenlemeniz gerekir. Ancak harici bir dosya kullanırsanız, sadece bu tek dosyayı değiştirmeniz yeterli olur.
- **Zaman Tasarrufu**: Değişiklik ihtiyacı duyulduğunda her sayfayı ayrı ayrı düzenlemek çok fazla zaman alacaktır. Harici dosya yapısı bu süreci hızlandırır.
- **Kod Düzeni (Temiz Kod):** Projenizdeki kod karmaşasından kurtulmanızı sağlar ve HTML yapınızın daha okunabilir kalmasına yardımcı olur.
- **Hata Tespiti:** Hataları ayıklamak ve tespit etmek merkezi bir dosya üzerinden çok daha kolaydır.
- **src Özniteliği Kullanımı**: Harici dosyalar her sayfaya <script src="..."> özniteliği kullanılarak kolayca dahil edilebilir.

Javascript

- Javascript kodlarını yazmak için Windows kullanıcıları için NotePad yeterlidir.
- Genellikle yazım tarzı şu şekildedir:

```
<script>
JavaScript kodlar
</script>
```

- Kullanılacak yere göre Html'in içerisinde kullanılır. Fakat genelde <head>...</*head*> etiketleri arasında kullanılır.
- Javascript kodları bittiğinde elinizde asla kendi başına çalışan uzantısı exe veya com olan bir dosya olmaz.
- Her zaman için tarayıcı tarafından yorumlanması gerekir. Yorumlanması demek Javascript kodunun çalışması anlamındadır.

<script language="javascript" src="“ornek.js" type="text/javascript"> </script>

JavaScript Söz Dizimi Kuralları

JavaScript'in söz dizimi C, C# ve Java dillerine benzerlik gösterir.

- **Noktalı Virgül (;):** JavaScript'te her bir komut satırı (deyim) ; ile bitmelidir. Örneğin: var i = 10;.
- **Büyük-Küçük Harf Duyarlılığı:** Dil yapısı gereği büyük ve küçük harfler birbirinden farklı algılanır.
- **Nokta (.) Operatörü:** Bir nesnenin özelliklerine veya metotlarına erişmek için kullanılır.
- **Açıklama Satırları (Comments):** Tarayıcı tarafından işlenmeyen, sadece geliştiriciye not bırakmak için kullanılan bölümlerdir.
  - **Tek satırlı açıklama:** // karakteri ile başlar.
  - **Çok satırlı açıklama:** /* ile başlar ve */ ile biter.

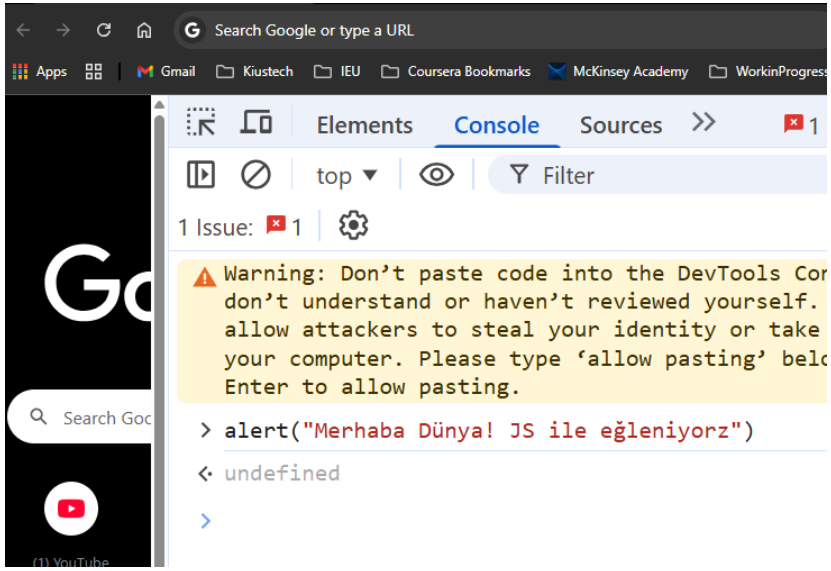
```
var araba = {
    marka: "Tesla",
    model: "Model 3",
    yil: 2024
};

// Nokta operatörü ile verilere erişim:
console.log(araba.marka); // Çıktı: Tesla
console.log(araba.yil);   // Çıktı: 2024
```

JavaScript Nerede Çalışır? (Mutfak: Browser Console)

- JavaScript'in en güzel yanı, hiçbir program kurmadan doğrudan **Tarayıcıda (Browser)** çalışmasıdır.
- Tarayıcı >> F12 (İncele) >> Network altında yüklene dosyaları inceleyerek JS dosyasını görüntüleyebilirsiniz.

```
alert("Merhaba Dünya! Ben artık bir programcıyım.");
```



Javascript-Alert İkaz Metodu

- Alert yöntemi, kullanıcıya bir uyarı mesajı ve bir OK düğmesi görüntülemek için kullanılır

```
alert ("Merhaba");
```

Javascript-Confirm Metodu

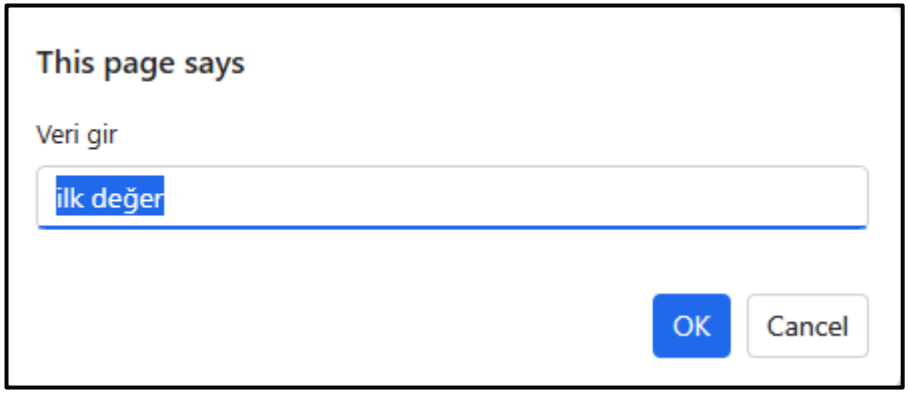
- Confirm yöntemi OK ve Cancel düğmelerine sahip bir mesaj görüntülemek için kullanılır. Ok basarsa True, Cancel' a basılırsa false döndürür.

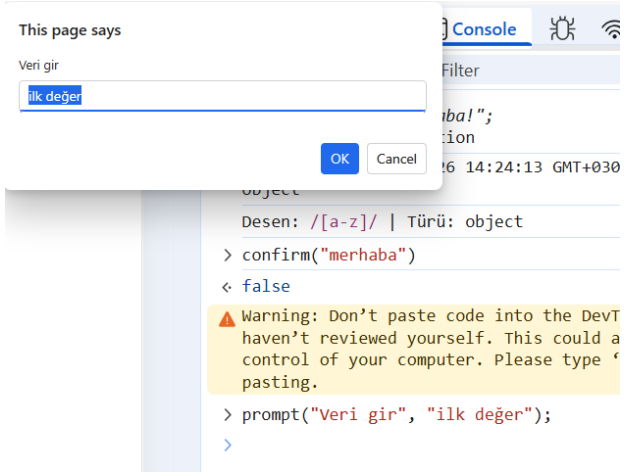
```
confirm("Onayliyormusunuz?")
```

Javascript-Prompt Metodu

- Kullanıcı girdilerini almak için bir iletişim kutusunu görüntüler.

```
prompt ("Veri gir","ilk deger");
```

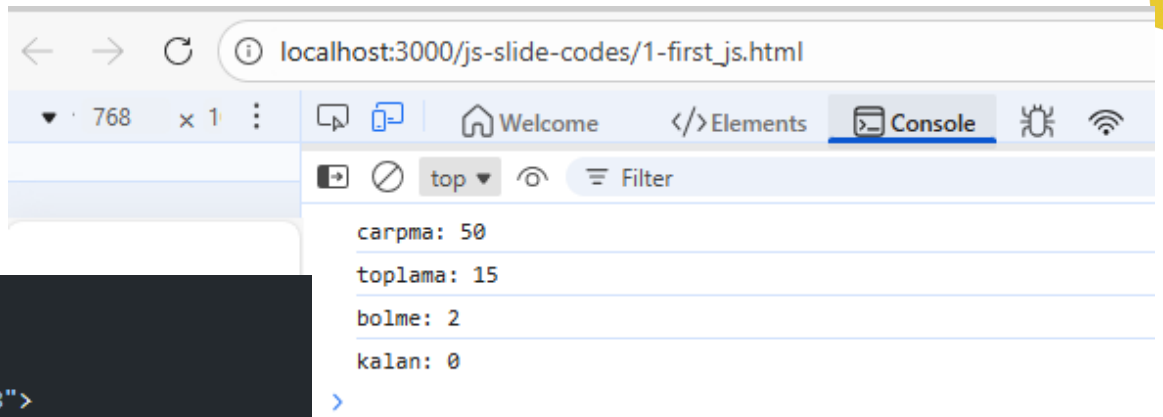




JavaScript

js-slide-codes >> 1-first_js.html

İlk Kod - Dahili




```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Konsoldan çıktı almak</title>
    <script type="text/javascript">
        var i = 10;
        var j = 5;

        var carpma = i * j;
        var toplama = i + j;
        var bolme = i / j;
        var kalan = i % j;

        console.log("carpma: " + carpma);
        console.log("toplama: " + toplama);
        console.log("bolme: " + bolme);
        console.log("kalan: " + kalan);
    </script>
</head>
<body>

</body>
</html>
```

Kodun Analizi

- **Değişken Tanımlama:** i değişkenine 10, j değişkenine ise 5 değeri atanmış.
- **Matematiksel İşlemler:**
  - **Çarpma (*):** $10 \times 5 = 50$
  - **Toplama (+):** $10 + 5 = 15$
  - **Bölme (/):** $10 / 5 = 2$
  - **Mod Alma (%):** 10'un 5'e bölümünden kalan 0
- **Konsol Çıktısı:** console.log() komutu, yanındaki metinle hesaplanan sonucu birleştirerek tarayıcının geliştirici araçları kısmına yazdırır.

KOD: js-react-ders-sunum-kodlari >> 1-ilk_kod.html

İlk Kod - Harici

JavaScript

```javascript
var i = 10;
var j = 5;

var carpma = i * j;
var toplama = i + j;
var bolme = i / j;
var kalan = i % j;

console.log("carpma: " + carpma);
console.log("toplama: " + toplama);
console.log("bolme: " + bolme);
console.log("kalan: " + kalan);
```

HTML

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Konsoldan çıktı almak</title>
    <script type="text/javascript" src="hesaplama.js"></script>
</head>
<body>
</body>
</html>
```

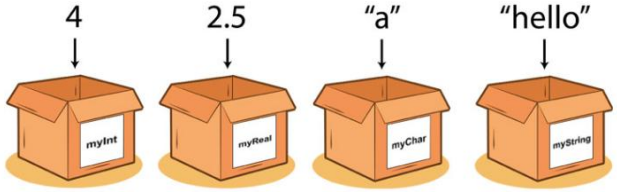
KOD: js-react-ders-sunum-kodlari>> 2-ilk_kod-harici.html

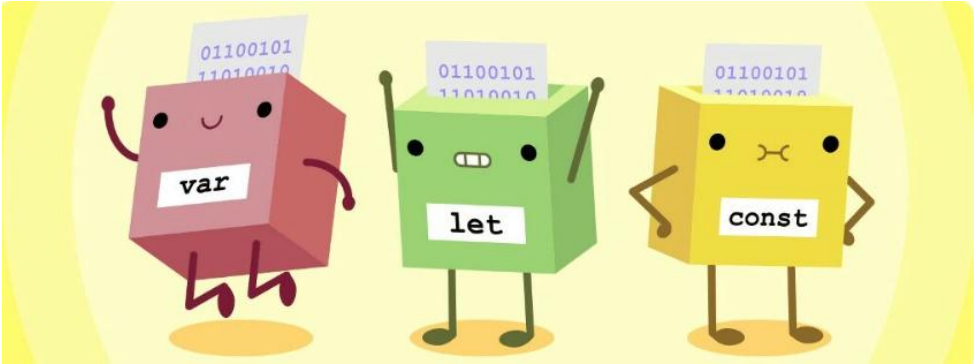
JS Temelleri

- Değişkenler
  - İsimlendirme
  - Kapsam
- Veri Türleri
- Tür Dönüşümleri
- Operatörler

Değişkenler:
Verinin "Saklama Kapları"

- Bilgisayarın hafızasında (RAM) bir yer ayırıp içine bir isim verip değer koymaktır.
- Bir taşınma kolisi gibi düşünün;
  - üzerine "Mutfak Eşyaları" yazarsınız (Değişken adı) ve
  - içine tabakları koyarsınız (Değişken değeri).

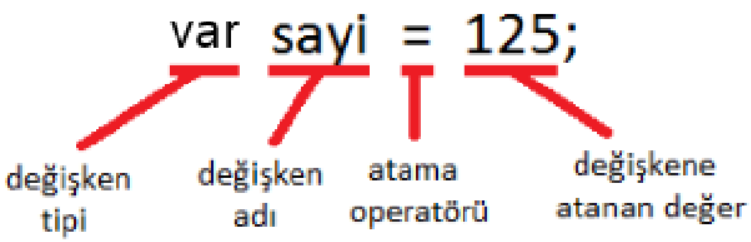




- **var**, değişken tanımlamak için kullanılan **eski** bir yöntemdir.
- **let**, değişken tanımlamak için kullanılan **yeni** bir yöntemdir.
- **const**, değişken değerinin sabit olduğunu belirtir ve herhangi bir **değişikliğe izin vermez**.

Değişken Adı Kuralları

```
var title = 'Code Life';
let publisher = 'House of Books';
const author = 'Melvin Gray';
```



- Değişken isimleri Türkçe karakter ***içermemelidir.***
- Değişken isimleri büyük ve küçük harf ***duyarlıdır.***
- Değişken isimlerinde ilk karakter ***bir sayı olamaz.***
- Değişken isimlerinde JavaScript etiketleri ***kullanılamaz.***
- Değişken isimlerinde sayı, harf, alt çizgi ve dolar işareti ***kullanılabilir;***
- Boşluk, noktalama işareti veya sembol ***kullanılamaz.***

JavaScript

Değişken isimlerinde JavaScript etiketleri kullanılamaz.

- break
- case
- catch
- continue
- debugger
- default
- delete
- do
- else
- finally
- new
- return
- switch
- for
- typeof
- function
- if
- in
- this
- throw
- try
- var
- void
- while
- with
- instanceof

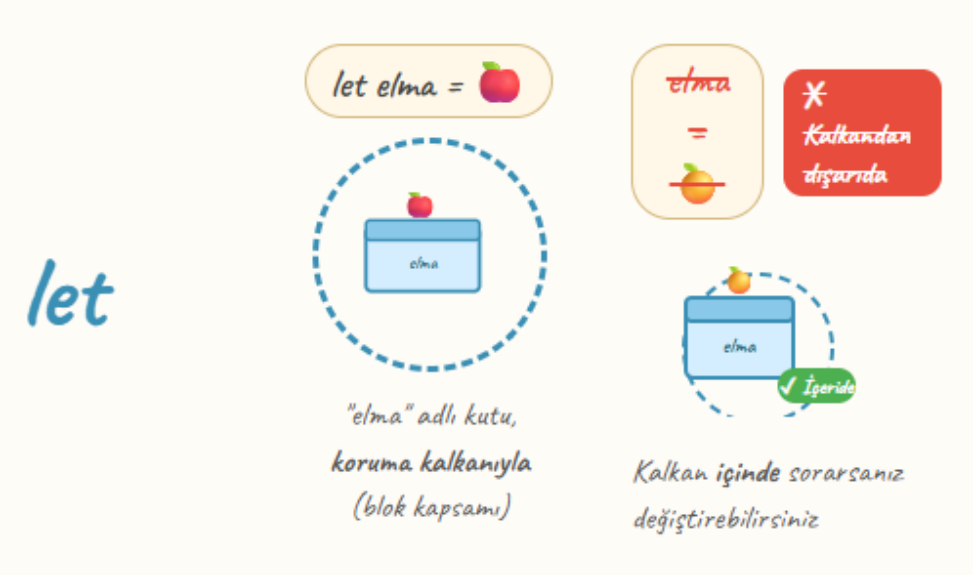
var

- JS'in ilk yıllarından beri var olan emektar komut.
- Dikkat edilmesi gereken bir tür!!!
- Bir değişkeni var ile tanımlarsanız, aynı isimle tekrar tanımlamanıza izin verir.
- **Kapsam (Scope):** var biraz gezentidir; bir fonksiyonun içinde değilse her yerden erişilebilir (Global Scope).

KOD: js-react-ders-sunum-kodlari>> 3-degiskenler.html

let

- Modern JavaScript (ES6) ile gelen, günümüzde en çok kullanılan değişken tanımlama yöntemidir.
- Aynı isimle ikinci kez tanımlama yapmanıza izin vermez, hata fırlatır.
- **Kapsam (Scope):** Sadece bulunduğu süslü parantezlerin { } içinde yaşar. Odadan dışarı çıkmaz.

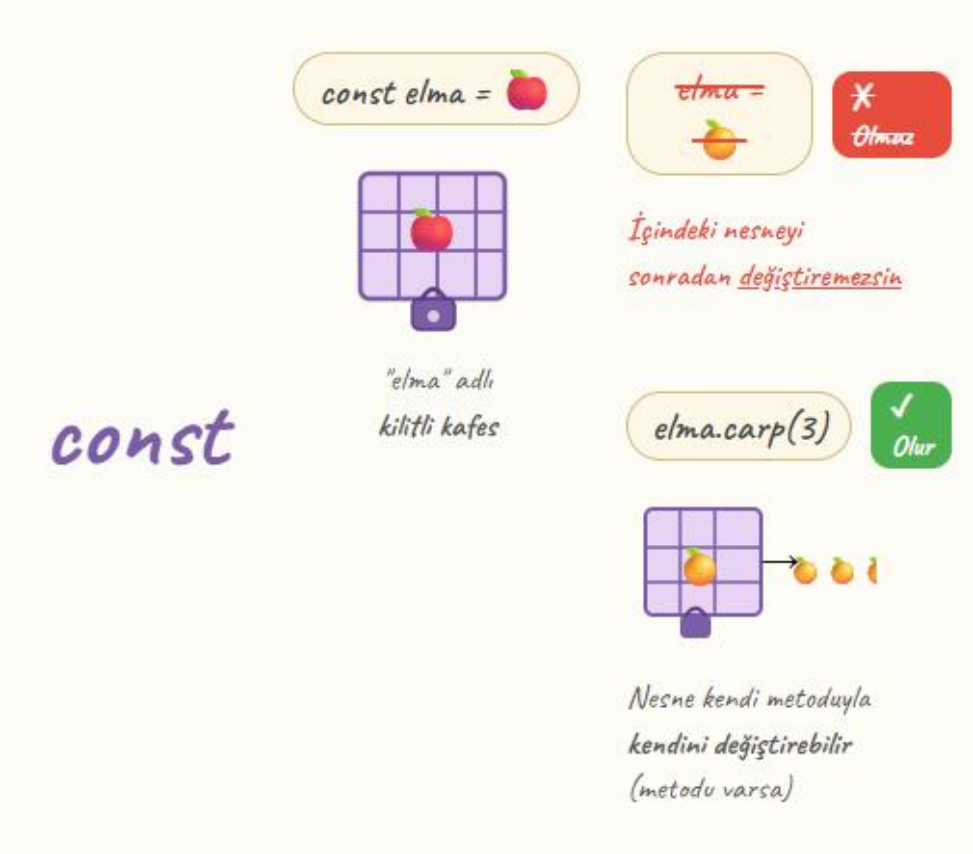


KOD: js-react-ders-sunum-kodlari>> 3-degiskenler.html

JavaScript

const

- Asla değişmez
- İngilizce "Constant" (Sabit) kelimesinden gelir.
- **Özelliği:** Bir kez değer atadınız mı, bir daha asla değiştiremezsiniz.
- **Kullanım Yeri:** Pi sayısı gibi sabitler, veritabanı bağlantı bilgileri gibi değişmesini istemediğimiz her yer.



KOD: js-react-ders-sunum-kodlari>> 3-degiskenler.html

var – let – const karşılaştırma

- **var**, değişken tanımlamak için kullanılan **eski** bir yöntemdir.
- **let**, değişken tanımlamak için kullanılan **yeni** bir yöntemdir.
- **const**, değişken değerinin sabit olduğunu belirtir ve herhangi bir **değişikliğe izin vermez**.

Özellik	`var`	`let`	`const`
Tekrar Tanımlama	Evet (Tehlikeli!)	Hayır	Hayır
Değer Güncelleme	Evet	Evet	Hayır
Modernlik	Eski (Kullanma)	Yeni (Standart)	Yeni (Sabitler İçin)

Dikkat!!! - "Hata Yaparsan Yanarsın"
Köşesi: Hoisting (Yukarı Çekme)

JavaScript

- **var** ile tanımlanan bir değişken, siz onu daha yazmadan kodun en başında varmış gibi davranabilir (Hoisting).
  - Ama **let** ve **const** duyarlı ve anlayışlıdır; onları yazmadan onlara seslenirseniz size "Tanımıyorum ki!" derler.
- **let** ve **const** kullanın!
  - var kapsamı çok geniş
  - var Yeniden değer verilmesi kontrolü zorlu
  - var Hoisting

KOD: js-react-ders-sunum-kodlari >> 3-degiskenler.html

JavaScript

VAR / LET Kıyaslaması

Özellik	var Anahtar Kelimesi	let Anahtar Kelimesi
Kapsam (Scope)	Fonksiyon kapsamlıdır (Function-scoped).	Blok kapsamlıdır (Block-scoped).
Yeniden Tanımlama	Aynı isimle tekrar tanımlamaya izin verir.	Aynı alan içinde tekrar tanımlamaya izin verilmez.
Başlatma (Initialization)	Başlangıç değeri atanmadan tanımlanabilir.	Kullanılmadan önce mutlaka başlatılması gerekir.
Döngü Kullanımı	Tüm fonksiyon boyunca paylaşılır.	Tanımlandığı blok ile sınırlandırılır.
Hoisting (Yukarı Taşıma)	Yukarı taşınır (Kapsamının en üstüne taşınmış gibi davranır).	Yukarı taşınmaz (Tanımlanana kadar "geçici ölü bölge" - temporal dead zone içinde kalır).
JavaScript Versiyonu	Bir ECMAScript1 özelliğidir.	ECMAScript6 (ES6) ile tanıtılmıştır.

Koddaki Hata nedir?

- const ile tanımlanan değer değiştirilemez.
- Değişken isimleri rakamla başlayamaz.
- String (metin) ifadeler tırnak içine alınmalıdır ("Merhaba...").

```
const okulAdi = "Ege MYO";
okulAdi = "Dokuz Eylül MYO";

let 1.Sira = "Ahmet";

var mesaj = Merhaba JavaScript;
```

MİNİ TEST – soru 1

JavaScript'te bir değişkenin değerinin program boyunca hiç değişmemesini istiyorsak hangi anahtar kelimeyi kullanmalıyız?

A. var

B. let

C. fixed

D. const

JavaScript

MİNİ TEST – soru 2

Örnekte geçen `let dersler = ["Programlama", "Veritabanı"];` tanımlaması hangi veri tipine örnektir?

A. Boolean

B. Object

C. Array (Dizi)

D. String

MİNİ TEST – soru 3

JavaScript

Bir öğrencinin mezun olup olmadığını belirten `let mezunMu = false;` değişkeni hangi veri tipindedir?

A. Null

B. Boolean

C. Number

D. Undefined

MİNİ TEST – soru 4

Aşağıdaki değişken tanımlamalarından hangisi JavaScript yazım kurallarına (syntax) göre hatalıdır?

A. let kullanici1 = "Veli";

B. let 1.kullanici = "Ali";

C. let $fiyat = 50;

D. let _sayi = 10;

MİNİ TEST – soru 5**

Bir değişken tanımlandığı halde ona henüz bir değer atanmamışsa, bu değişkenin otomatik olarak aldığı veri tipi hangisidir?

A. 0

B. null

C. empty

D. undefined

MİNİ TEST – soru 6**

Aşağıdaki kod parçasının çıktısı ne olur?

```
let x = "5";
let y = 2;
console.log(x + y);
```

A. 7

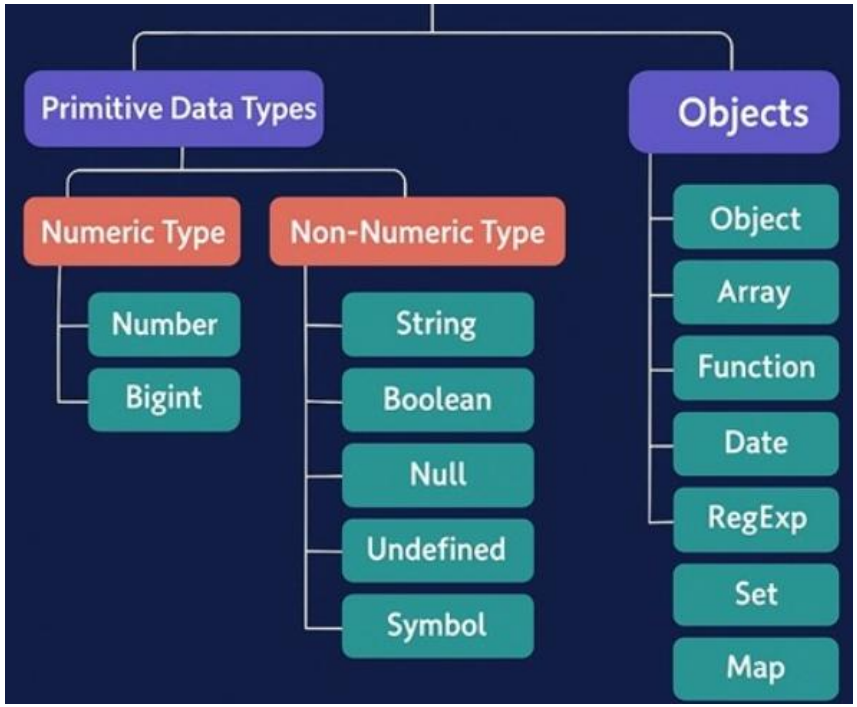
B. Hata verir

C. "52"

D. NaN

Veri Tipleri: "Kutunun İçinde Ne Var?"

- JavaScript'te veri tipleri, kolinin içine koyduğumuz eşyanın **cinsi** gibidir.
- JavaScript "dinamik" bir dildir.
  - **Yani bir değişkenin içine önce bir sayı, sonra bir metin koymanıza pek kızmaz (ama biz yine de düzenli olalım!).**



Veri Tipleri

İlkel Veri Tipleri (Primitive Data Types)

Tek bir değer tutarlar ve değiştirilemezler (immutability).

- **Sayısal Tipler (Numeric)**
  - **Number:** Her türlü sayı. 10, 3.14, -5. JavaScript'te tam sayı/ondalıklı sayı ayrımı yoktur, hepsi "Number"dır.
  - **BigInt:** Çok ama çok büyük sayılar için kullanılır. Normal Number tipinin kapasitesini aşan (2^53'ten büyük) sayılarda sonuna n ekleyerek kullanılır.
- **Sayısal Olmayan Tipler (Non-Numeric)**
  - **String:** Metinler. Tırnak içindeki her şey: "Merhaba Dünya".
  - **Boolean:** Mantıksal anahtarlar. Sadece true veya false.
  - **Null:** "Bilerek boş bırakılmış" demektir. Bir değişkenin değerinin boş olduğunu kasten belirtmek için biz atarız.
  - **Undefined:** "Henüz tanımlanmamış" demektir. Bir değişken oluşturulmuş ama içine değer konmamışsa JS otomatik olarak bunu atar.

KOD: js-react-ders-sunum-kodlari>> 4-veri_turleri.html

Nesneler (Objects) –

İçlerinde birden fazla değer ve hatta fonksiyon barındırabilirler.

- **Object (Nesne):** { anahtar: değer } şeklinde çalışan temel yapı.
  - { ad: "Volkan", yas: 40 }.
- **Array (Diziler):** Listelerdir. Köşeli parantez içinde tutulur.
  - ["Elma", "Armut", "Muz"].
- **Function (Fonksiyonlar):** Aslında JS'de fonksiyonlar da birer objedir! Kod bloklarını saklar ve istediğimizde çalıştırır.
- **Date:** Tarih ve saat bilgilerini tutan özel nesneler.
- **RegExp:** Düzenli ifadeler. Metinler içinde karmaşık arama ve değiştirme (örn: e-posta kontrolü) yapmak için kullanılır.
- **Set:** İçinde aynı değerden sadece bir tane barındıran özel listeler. (Tekrar eden eleman istemediğimizde kullanırız).
- **Map:** Objeye benzer ama anahtar kısmına sadece metin değil, her şeyi koyabildiğimiz gelişmiş sözlük yapılarıdır.

Veri Tipleri

İlkel Veri Tipleri (Primitive Data Types)

Tek bir değer tutarlar ve değiştirilemezler (immutability).

- Sayısal Tipler (Numeric)
  - **Number:** Her türlü sayı. 10, 3.14, -5. JavaScript'te tam sayı/ondalıklı sayı ayrımı yoktur, hepsi "Number"dır.
  - **BigInt:** Çok ama çok büyük sayılar için kullanılır. Normal Number tipinin kapasitesini aşan (2^53'ten büyük) sayılarda sonuna n ekleyerek kullanılır.
- Sayısal Olmayan Tipler (Non-Numeric)
  - **String:** Metinler. Tırnak içindeki her şey: "Merhaba Dünya".
  - **Boolean:** Mantıksal anahtarlar. Sadece true veya false.
  - **Null:** "Bilerek boş bırakılmış" demektir. Bir değişkenin değerinin boş olduğunu kasten belirtmek için biz atarız.
  - **Undefined:** "Henüz tanımlanmamış" demektir. Bir değişken oluşturulmuş ama içine değer konmamışsa JS otomatik olarak bunu atar.

KOD: js-react-ders-sunum-kodlari>> 4-veri_turleri.html

Veri Tipleri

KOD: js-react-ders-sunum-kodlari>> 4-veri_turleri.html

Nesneler (Objects) –

İçlerinde birden fazla değer ve hatta fonksiyon barındırabilirler.

- **Object (Nesne):** { anahtar: değer } şeklinde çalışan temel yapı.
  - { ad: "Volkan", yas: 40 }.
- **Array (Diziler):** Listelerdir. Köşeli parantez içinde tutulur.
  - ["Elma", "Armut", "Muz"].
- **Function (Fonksiyonlar):** Aslında JS'de fonksiyonlar da birer objedir! Kod bloklarını saklar ve istediğimizde çalıştırır.
- **Date:** Tarih ve saat bilgilerini tutan özel nesneler.
- **RegExp:** Düzenli ifadeler. Metinler içinde karmaşık arama ve değiştirme (örn: e-posta kontrolü) yapmak için kullanılır.
- **Set:** İçinde aynı değerden sadece bir tane barındıran özel listeler. (Tekrar eden eleman istemediğimizde kullanırız).
- **Map:** Objeye benzer ama anahtar kısmına sadece metin değil, her şeyi koyabildiğimiz gelişmiş sözlük yapılarıdır.

Null vs Undefined

Null

- Bir kutu var, içini özellikle biz boşalttık ve "burası şimdilik boş kalsın" dedik (Kutunun içine "BOŞ" etiketi koyduk). `null`

Undefined

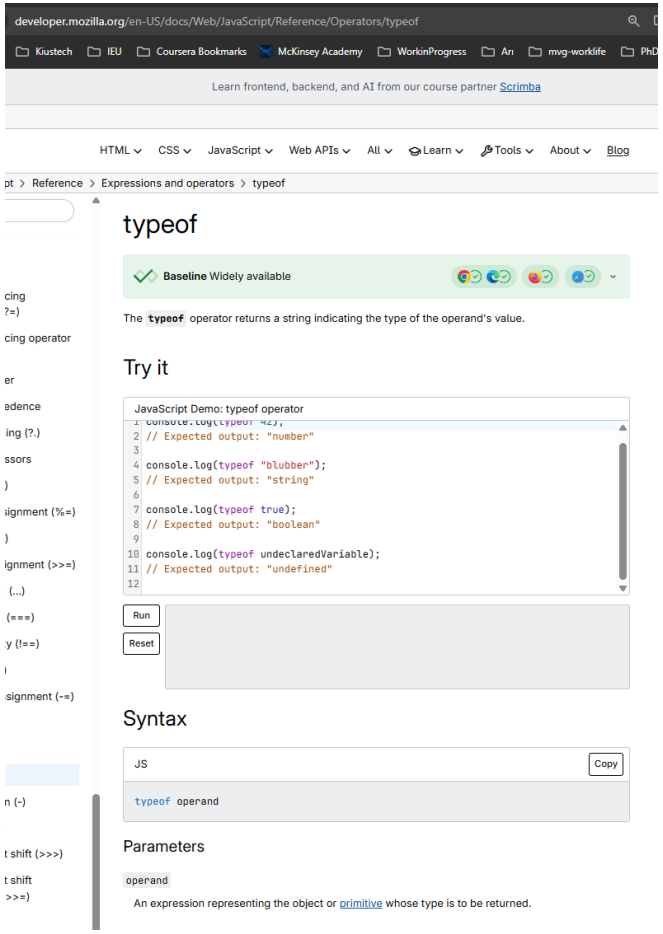
- Bir kutu var ama içi boş, henüz bir şey koymadık (Kutuyu açtın, boş çıktı).

`undefined`

KOD: js-react-ders-sunum-kodlari>> 4-veri_turleri.html

Veri Türünü Tespit

- **Typeof()**
  - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/typeof
  - işlenenin (operand'ın) değerinin türünü belirten bir string (metin) döndürür.



KOD: js-react-ders-sunum-kodlari>> 4-veri_turleri.html

ÖDEV 1

SORU 1:

Javascript string literals (kaçış karakterleri – escape characters) nedir ve nasıl kullanılır?

SORU 2:

'…' (tek tırnak), "…" (çift tırnak), `…` (Template Literals) farkı nedir? Örnek ile açıklayınız.

SORU 3:

Number(sayı) veri tipinin max_value ve mi_value değerlerini nasıl elde edersiniz? Bu değerleri tipi sayı olan farklı değişkenlere atayarak "alert" metodu ile yazdırın.

SORU 4: Aşağıdaki kodu JS-HTML ile yazınız:

Adınızı, soyadınızı, yaşınızı, bölümünüzü farklı değişkenlere metin(string) olarak atayın.

Mezuniyet durumunuzu "mezun" isimli bir değişkene mantıksal(Boolean") olarak atayın.

Ekrana

Merhaba, ben …ad soyad…

Yaşım: ……

Bölümüm: ……

Mezunsanız "Mezunoldum!"

Değilseniz "Hala öğrenciyim." olarak yazdırın

İlkel Veri Tipleri (Primitive Data Types) Örnekleri

JavaScript

KOD: js-react-ders-sunum-kodlari
>> 4-veri_turleri.html

```
// --- Sayısal (Numeric) ---
let sayi = 35;
console.log("Değer:", sayi, "| Türü:", typeof sayi); // number

let buyukSayi = 9007199254740991n;
console.log("Değer:", buyukSayi, "| Türü:", typeof buyukSayi); // bigint

// --- Sayısal Olmayan (Non-Numeric) ---
let metin = "İzmir Dokuz Eylül";
console.log("Değer:", metin, "| Türü:", typeof metin); // string

let dogruMu = true;
console.log("Değer:", dogruMu, "| Türü:", typeof dogruMu); // boolean

let bos = null;
console.log("Değer:", bos, "| Türü:", typeof bos); // object* (Dikkat: JS'

let tanimsiz;
console.log("Değer:", tanimsiz, "| Türü:", typeof tanimsiz); // undefined
```

KOD: js-react-ders-sunum-kodlari
>> 4-veri_turleri.html

JavaScript

Nesneler (Objects) Örnekleri

```
// Standart Obje (Anahtar-Değer Çifti)
let hoca = { ad: "Volkan", meslek: "Solution Architect" };
console.log("Obje:", hoca, "| Türü:", typeof hoca); // object

// Array (Dizi / Liste)
let diller = ["JavaScript", "PHP", "Java"];
console.log("Dizi:", diller, "| Türü:", typeof diller); // object (Diziler de

// Function (Fonksiyon)
let selamVer = function() { return "Merhaba!"; };
console.log("Fonksiyon:", selamVer, "| Türü:", typeof selamVer); // function

// Date (Tarih)
let bugun = new Date();
console.log("Tarih:", bugun, "| Türü:", typeof bugun); // object

// RegExp (Düzenli İfade)
let desen = /[a-z]/;
console.log("Desen:", desen, "| Türü:", typeof desen); // object
```

Set ve Map

KOD: js-react-ders-sunum-kodlari >> 4-veri_turleri.html

JavaScript

```
console.log("-----SET-----");
let set = new Set();
set.add("değer1");
set.add("değer2");
set.add("değer3");
console.log("Set:", set, "| Türü:", typeof set);
set.add("değer2"); /* Set'e aynı değeri eklemeye çalıştığımızda,
 zaten var olduğu için eklenmez.*/
console.log("Set (Tekrar ekleme denemesi sonrası):", set);

console.log("-----MAP-----");
let map = new Map();
map.set("key1", "value1");
map.set("key2", "value2");
map.set("key3", "value3");
console.log("Map:", map, "| Türü:", typeof map);
console.log("Map'ten key2'nin değeri:", map.get("key2")); // value2
```

JavaScript

“typeof null” Tuzağı

- JS ilk sürümlerinden kalan bir hata nedeniyle **object** döner. Ama biz biliyoruz ki null aslında bir **Primitive** (İlkel) veri türüdür. Yani bu bir "yazılımsal hata" klasiğidir!

```
let nullDeger = null;
console.log("Değer:", nullDeger, "| Türü:", typeof nullDeger);
```

Uygulamada: js-react-ders-sunum-kodlari >> veri_tipleri_html.html

KOD: js-react-ders-sunum-kodlari>> 4-veri_turleri.html

JavaScript

Sınıf Çalışması

- Kendi isim, soyad, yaş ve öğrenci olup olmadığınızı içeren bir ***ogrenci*** objesi oluşturun ve **console.log** ile yazdır

```
let ogrenci = {
    isim: "Mustafa",
    yas: 20,
    aktifMi: true
};
console.log(ogrenci);
```

```
▼ {isim: 'Mustafa', yas: 20, aktifMi: true} i
    aktifMi: true
    isim: "Mustafa"
    yas: 20
  ▶ [[Prototype]]: Object
```

Tür Dönüşümleri – Dikkat!!!

Açık Dönüşüm (Explicit Conversion)

- **String'e Dönüştürme:** Bir sayıyı veya boolean değeri metne çevirmek için **String()** kullanılır.
- **Sayıya Dönüştürme:** Metin içindeki sayıları matematiksel işleme sokabilmek için **Number()** kullanılır.

Kapalı/Otomatik Dönüşüm (Implicit Conversion)

- JavaScript'te program kodlarının akışında değişkenin veri türü, içerisine atanan değere göre örtülü olarak (*otomatik olarak*) değişebilir
- Type Coercion

Tür Dönüşümleri - Açık Dönüşüm

JavaScript

Açık Dönüşüm (Explicit Conversion)

- **String'e Dönüştürme:** Bir sayıyı veya boolean değeri metne çevirmek için **String()** kullanılır.

```
let sayi = 35;
let metinHali = String(sayi);
console.log(typeof metinHali);
```

- **Sayıya Dönüştürme:** Metin içindeki sayıları matematiksel işleme sokabilmek için **Number()** kullanılır.

```
let fiyat = "150";
let gercekSayi = Number(fiyat);
console.log(gercekSayi + 50); /
```

Tür Dönüşümleri –
Otomatik Dönüşüm (Type Coercion)

Kapalı/Otomatik Dönüşüm
(Implicit Conversion)

- **Toplama İşlemi (+):** Eğer taraflardan biri String ise, JS diğerini de metne çevirip yan yana yapıştırır.
  - 5 + "5" ➔ "55" (Metin!)
- **Çıkarma, Çarpma, Bölme (- , * , /):** JS burada "Metinle matematik olmaz, ben bunu sayıya çevireyim" der.
  - "10" - 2 ➔ 8 (Sayı!)
- JavaScript'te program kodlarının akışında değişkenin veri türü, içerisine atanan değere göre örtülü olarak (*otomatik olarak*) değişebilir.
- *Kodlamada dikkat edilmelidir.*

Tür Dönüşümleri – NaN (Not a Number)

JavaScript

- Eğer JS'e imkansız bir dönüşüm yaptırmaya çalışırsanız (örneğin "Merhaba" kelimesini sayıya çevirmek), size küser ve **NaN** sonucunu verir.

```
let sonuc = Number("Selam");
console.log(sonuc); // NaN
console.log(typeof sonuc); /
```

```
> let sText = "20";
  console.log(Number(sText));
  20                                  VM281:2
```

```
> let s2Text = "2deli0";
  console.log(Number(s2Text));
  NaN                                 VM298:2
```

Tür Dönüşümleri - Mantıksal Dönüşüm (Boolean Conversion)

JavaScript

- Bir değeri true veya false yapma işlemidir.
- JS'de her şeyin bir "doğruluk" değeri vardır.

```
> console.log(Boolean("merhaba"));
true                                VM392:1
```

!

```
> console.log(Boolean(" "));
true                                VM406:1
```

```
> console.log(Boolean(""));
false                               VM413:1
```

- **Falsey (Yalancı) Değerler:** 0, "" (boş string), null, undefined, NaN. Bunlar Boolean() içine girince her zaman false çıkar.
- **Truthy (Gerçekçi) Değerler:** Geri kalan her şey (dolu metinler, 0 hariç sayılar, objeler) true çıkar.

```
console.log(Boolean(1));      // true
console.log(Boolean(0));      // false
console.log(Boolean(""));     // false
console.log(Boolean("JS"));   // true
```

Haydi Cevapla – Soru1

Kodun çıktısı nedir ve neden bu çıktı oluşur?

```
<script>
  let x = "5";
  let y = 5;
  let z = x + y;

  console.log(typeof z);
  console.log(z);
  // Soru: Çıktı nedir ve neden?
</script>
```

Sorular - Verilen kod parçalarını çıktıları nedir?

```
1. console.log(true + 1);
2. console.log("" + 0);
3. console.log( Number(12.5) );
4. console.log( Boolean("False"));
5. console.log( Boolean(true));
```

Tür Dönüşüm fonksiyonları

- **parseInt():**
  - Metin (String) içindeki bir ifadeyi tarar ve bulduğu ilk **tam sayıyı** (Integer) çekip çıkarır. Eğer metin bir sayı ile başlıyorsa, arkasındaki yazıları görmezden gelir.
  - **Kural:** Metin mutlaka bir rakamla başlamalıdır. Harfle başlarsa küser ve NaN döner.

```
let veri1 = parseInt("40");       // Sonuç: 40
let veri2 = parseInt("40.99");    // Sonuç: 40
let veri3 = parseInt("100px");    // Sonuç: 100
let veri4 = parseInt("Hata 50"); // Sonuç: NaN
```

- parseFloat():
  - Metin içindeki **ondalıklı sayıları** (Float) olduğu gibi koruyarak çekip alır. Noktadan sonrasını çöpe atmaz.
  - **Kullanım Yeri:** Fiyat hesaplamaları, koordinatlar veya hassas ölçümlerde vazgeçilmezdir.

```
let fiyat = parseFloat("19.99 TL"); // Sonuç: 19.99
let derece = parseFloat("36.6 derece"); // Sonuç: 36.6
let tamSayi = parseFloat("50"); // Sonuç: 50 (Nokta yok
```

Tür Dönüşüm fonksiyonları

- **.toString():**
  - Elinizdeki bir sayı veya boolean değeri metne (String) dönüştürmek için kullanılır.
  - . İle method olarak çağırılır.
  - **Özelliği:** Sayısal bir değerle metni birleştirmeden önce "garantiye almak" için kullanılır.

```
let sayi = 123;
let metin = sayi.toString();
console.log(typeof metin); // "string"

let cevap = true;
console.log(cevap.toString()); // "true"
```

Number() vs parseInt()

- Number("100px") ➔ **NaN** döner (Çünkü içinde 'px' var, "bu sayı değil" der).
- parseInt("100px") ➔ **100** döner (Çünkü içinden sayıyı "ayıklamayı" bilir).

Veri temizse **Number(),**

veri kirliyse (yanında birim vs. varsa) **parseInt()** kullanılır.

JavaScript

Uygulama

Kullanıcıdan doğum yılı bilgisini alan ve yaşını hesaplayan uygulama hazırlayınız.

```
// Kullanıcıdan doğum yılını al (string gelir)
let dogumYili = prompt("Doğum yılınızı giriniz:");

// String'i number'a çevir
dogumYili = parseInt(dogumYili);

// Şu anki yılı al
let simdikiYil = new Date().getFullYear();

// Yaşı hesapla
let yas = simdikiYil - dogumYili;

// Sonucu yazdır
console.log("Yaşınız: " + yas);
```

Soru

Elimizde "150.50TL" ve "25.20TL" şeklinde iki string var. Bunları toplayıp ekrana 175.7 yazdıran kodu yazın.

```
let fiyat1 = "150.50TL";
let fiyat2 = "25.20TL";

// TL kısmını silip sayıya çeviriyoruz
let sayi1 = parseFloat(fiyat1);
let sayi2 = parseFloat(fiyat2);

let toplam = sayi1 + sayi2;

console.log(toplam); // 175.7
```

Operatörler

JavaScript

- Değişkenler ve değerler üzerinde işlem yapmamızı sağlayan özel sembollerdir.
- **Görevleri:** Sayıları toplamak, metinleri birleştirmek veya iki değeri birbiriyle kıyaslamak ve birçok süreç için kullanılırlar.

- Aritmetik Operatörler
- Atama Operatörleri
- Karşılaştırma Operatörleri
- Mantıksal Operatörler
- Bitsel (Bitwise) Operatörler
- Özel Operatörler
  - Koşul (Ternary) Operatörü

Operatör Türleri

KOD: js-react-ders-sunum-kodlari
>> operatorler.html

Aritmetik Operatörler

+ Toplama
- Çıkarma
***** Çarpma
/ Bölme
% bölmeden sonra kalan (Örneğin: 12%5 işlemi 2 değerini verir.)
++ Bir artırma
-- Bir azaltma.

Atama Operatörleri

= Atama
+= Topla ve Ata
-= Çıkar ve Ata
*= Çarp ve Ata

Karşılaştırma Operatörleri

== İstenenler eşitse true döndürür
!= Eşit değil
> Büyüktür
< Küçüktür
>= Büyük Eşit
<= Küçük Eşit
=== Denk
!== Denk değil

Mantıksal Operatörler

&& ve "And"
İki ifade de doğruysa True döndürür onun dışında false döndürür

|| veya - "Or"
Ifadelerden birisi doğruysa True döndürür. Sadece iki ifade de false oldugunda false dödürür.

! Degil - "Not"
Negatifleştirme işlemi doğruysa True döndürür. true ise false, false ise true yapar

Koşul (Ternary) Operatör

JavaScript

koşul ? doğruysa_bu_çalışır : yanlışsa_bu_çalışır;

- Soru işareti varsa karar veriliyor demektir.
- If-else yapısınıntek satıra indirilmiş operatör halidir.
- **Görevi:** Bir koşulu kontrol eder; koşul doğruysa bir işlem, yanlışsa başka bir işlem yapar.

```
let saat = 20;
let selam = (saat < 18) ? "İyi Günler" : "İyi Akşamlar";
console.log(selam);
```

KOD: js-react-ders-sunum-kodlari >> operatorler.html

JavaScript

Operatörlerde Öncelik

Operatörler
. [ ] *[En Yüksek Öncelik]*
( ) new
! ~ - + ++ -- typeof void delete
* / %
+ -
<< >> >>>
< <= > >= in instanceof
== != === !==
&

Operatörler
^
|
&&
||
?:
= += -= *= /= %= <<= >>= >>>= &= ^= |=
, *[En Düşük Öncelik]*

Kontrol İfadeleri

- If
- If-Else
- If … Else – If
- İç İçe If yapıları
- Switch – Case

Akış

- Kontrol İfadeleri
- Döngüler
- Diziler

Javascript-Sayfaya yazı yazmak

- JavaScript kullanarak ekrana yazı yazdırmak document.write komutu ile yapılır.

Kullanım şekli:

```
document.write ("goruntulenmek istenenler" , degisken_ismi );
```

Değişkenler çift tırnak içerisinde yazılmazlar.
Sadece görüntülenmek istenenler çift tırnak içerisinde yazılır

Okumalar

- **JS Mutable ve Immutable Kavramları:** https://medium.com/@ie.turkmen/javascripttte-mutable-ve-immutable-kavramlar%C4%B1-anlamak-ve-kullanmak-1c90013aeea0
- Sembol türü:**JavaScripte Neden Symbol Türüne İhtiyaç Duyuldu ?** https://medium.com/frontend-development-with-js/neden-javascripte-neden-symbol-t%C3%BCr%C3%BCne-i%CC%87htiya%C3%A7-duyuldu-8f010abd621a